Group 53 Progress Report:

Kelsey Rough, Rishi Dhanwantari, Rhea Gokhale
{roughk,dhanwanr,gokhaler}@mcmaster.ca

1 Introduction

The Swing Trading Strategy Analyzer tests if machine learning can help decide whether a setup is strong or weak for the moving-average pullback strategy specifically. A rule-based detector scans 4-hour bars across 76 stocks. It flags a setup when a stock is trending above the 50-bar simple moving average (SMA), then pulls back into its 20-bar exponential moving average (EMA), and then continues moving up.

The machine learning model gives each setup a probability of being successful. Every trade has a stop loss $1.5\times$ the average true range (ATR) under the entry and a profit target $3\times$ the ATR above it, with a 10-bar time limit. A success means the target is reached before the stop loss is hit and before the time limit is over. The model uses price action features and macro features influenced by the overall market. Features include news tone from GDELT and the VIX volatility index.

This problem is difficult for 3 reasons:

1. There is a lot of noise in the market, so models can be wrong more often than not.

2. Financial machine learning results can be inflated by leakage, where a feature accidentally uses information from after the trade decision.

3. News data comes from a different source than the price data, so it has to be lined up carefully in time.

Due to this, we are not aiming for a super accurate model. We are aiming for one that is correct and does not leak. The project is a success if filtering setups with the model gives better trading results than taking every setup the detector finds or than just buying and holding.

2 Related Work

Krauss et al. (2017) used neural networks, gradient-boosting trees and random forest to predict which S&P 500 stocks would do better than the median the next day, based on past returns. Gradient-boosting trees performed similar to the other two methods, and results shrank in later years and with transaction costs. Like ours, their project uses tree-based methods to screen a large group of liquid stocks for short-term trades, but our model is tested across several time windows rather than one test period.

In our work, we modified the triple barrier model made by López de Prado (2018), which classifies based on whether they reach a target, stop-loss, or time barrier. The book explains that k-fold cross-validation often has leaks in financial datasets due to the dependency between labels of overlapping transactions, so we use walk-forward splitting to train our model on historical data only. Our project is also different in that we use a rule detector and meta-labeling rather than having the model select long trades directly.

Zhang et al. (2025) investigated the use of machine learning to identify customers who are likely to leave a bank. It evaluated eleven different machine learning models, including XGBoost, CatBoost, and LightGBM, using various data split ratios and sampling methods to address class imbalance. The research emphasized recall as its primary evaluation metric and identified numerous important predictors of churn. Like our project, it uses supervised machine learning, feature engineering, and gradient-boosting algorithms to predict binary outcomes while addressing class imbalance, but the two differ overall in terms of objective and application.

Basak et al. (2019) used random forests and XGBoost to predict whether a stock's price would rise or fall compared to its price a certain number of days earlier. Technical indicators like RSI and MACD were used as features, and the study showed high accuracy for medium to long-term

predictions. Like our project, they used technical indicators as inputs to tree-based models, but we use them for individual pullback setups on 4-hour bars rather than to fixed-horizon direction, and our model did not find a reliable signal.

Tetlock (2007) measured the pessimism of a daily Wall Street Journal column and found that high pessimism predicted lower market prices, and then a return to normal levels. The study concluded that media tone reflects investor sentiment rather than actual new information about the value of companies. It uses the tone of news as a signal for market movement, whereas we use GDELT tone across many news sources rather than a single column. We use it as a feature for classifying trades instead of for for predicting market returns directly.

3 Dataset

Data sources: We use hourly OHLCV data from Yahoo Finance, the yfinance library for 76 US stocks, and SPY and QQQ as benchmark indices. The 76 tickers are split into three groups – group 1 consists of technology stocks, group 2 is media, real-estate, and industrial businesses, and group 3 has finance, energy and healthcare stocks. We have extracted the maximum amount of data points, which includes a 665-day window with a 60-day warmup for the indicators. We also get macro data from two other sources: Yahoo's daily VIX closes and the GDELT 2.0 database, which is compiled into hourly news counts and tone using Google BigQuery.

Preprocessing: We store hourly bars locally, then resample them to two bars per trading day, one in the morning and one in the afternoon, while dropping any bars that are still in progress. To make sure our data stays clean, we flag invalid OHLCV values and remove any price jumps of over 20% or days with only one bar.

Setup detection: Each setup is a long moving average pullback, detected using a rule-based detector on the 4-hour bars. To be considered a valid setup, the following must be present:

- An uptrend, where the close and EMA20 are both above the SMA50 and the SMA50 is ascending in the last 10 bars
- A pullback, wich RSI(14) is ≤ 55 and where the recent low is within 0.5% of the EMA20 in the last 3 bars
- A bounce, with the signal bar being above EMA20 and above the previous bar's high.

A trade entry is done on the next bar's opening, and there can only be one position per ticker, so new setups are ignored until the current trade is complete.

	Group 1	**Group 2**	**Group 3**	**All**
Tickers	26	25	25	76
Setups	460	382	472	1,314
Hit target	24.6%	15.4%	24.5%	21.9%
Profitable	48.5%	42.7%	47.0%	46.6%
Avg R per trade	0.233	0.022	0.174	0.151

Table 1: Detector setups and outcomes for ticker groups

Labeling: Setups are then labeled using the triple-barrier technique (López de Prado, 2018). A stop is put 1.5×ATR below the entry, while the target is 3×ATR above it without exceeding 10 bars or 5 days in length. Setups are considered wins (1) if the target is hit, while time exits and stop losses are counted as losses (0). Incomplete trades are not labelled and excluded from the model training dataset. Every trade returns a value of $R = (\text{exit} - \text{entry})/(\text{entry} - \text{stop})$, which will be used in backtesting. If both the stop and the target are reached in one 4-hour bar, the winner is determined by the hourly bar. However, this was not necessary for our sample.

Dataset properties: A total of 1,314 setups were generated from the 76 tickers for the period of January 13, 2025 to November 2, 2026. Out of the 1,314 setups, 1,306 setups were labeled, and only 8 were open. Out of the 1,306 setups that were labeled, 21.9%, 44.6% and 33.5% achieved their target, stopped or expired, respectively, showing that some classes are imbalanced. Overall, 46.6% of trades were profitable, with an average profit/loss per trade of $+0.151$R and the median profit/loss is -0.19R. The average duration of a trade is 6.8 bars, and 14.8% of setups triggered during the afternoon bar. Group 2 has relatively poor performance compared to other groups (Table 1). In order of signal time, the setups are sorted into 1,038 training setups and 268 test setups (June 23 through October 30, 2026).

4 Features

Each setup is one row of data with 21 features, which are put into tree models, without embeddings or neural networks. Stock prices vary a lot, so each

is converted into a percentage or ratio so that all tickers are on the same scale.

Detector features:

- RSI
- ATR is percent of price
- how far the close price is above EMA20
- how far the close price is above SMA50
- trend strength is how far the EMA20 is above the SMA50
- if the signal came on the morning or afternoon bar

Technical features: These add information the detector doesn't use:

- MACD line and histogram divided by the close price
- change in the MACD histogram over the 3 pullback bars, where positive means momentum is turning upward
- 20-bar volatility of log returns
- 5-bar vs. 20-bar volatility ratio, and above 1 means volatility is getting more intense
- signal bar volume compared to the average of the last 20 bars from the same session (the morning bar is 4 hours and the afternoon bar is 2.5 hours, so they can't be mixed)
- average relative volume during the pullback where low means a quiet dip and not heavy selling

Macro features: Overall market tone when the signal fires:

- VIX: latest close, its 5-day change, and its distance from its 20-day average
- GDELT, from 72 hours before the signal bar closes:
  - average news tone
  - average Goldstein score (a scale from conflict to cooperation)
  - average tone of U.S. events
  - change in tone (last 24 hours vs. the last 48 hours)
  - news volume ratio (last 24 hours vs. the last 48 hours)

Leakage prevention:

- Each feature table saves the timestamp of the newest data it used, and a check makes sure none of them are later than the signal bar's close.
- The VIX daily close only counts as available from the next day.
- A GDELT hour only counts once it is fully over.
- We also run a truncation test. It cuts the raw data off right after random setups, rebuilds the features, and checks that values for earlier setups do not change. If they did, the feature would be using future data.
- Separate checks were run for the technical and macro features.

Feature selection. We also tested relative strength features against SPY: a 20-bar relative return, a short-term relative return, and a rank across tickers. They didn't improve the walk-forward results, so we left them out. The effect of each feature group is shown in the ablation in the Implementation section.

5 Implementation

Model Description: For our model, we use XGBoost and LightGBM. This section will primarily focus on XGBoost (for clarification on the status of LightGBM, please see the next paragraph). XGBoost, which is a gradient-boosted decision tree machine learning framework, is used to predict whether each moving-average pullback setup is a win or a loss. Every setup has 21 features: 6 detector features (RSI, ATR, distances to EMA and SMA, trend gap, and time of day), 7 technical features (derived from MACD, volatility, and volume), and 8 macro features (derived from GDELT and VIX news tone). The training loss is the logistic loss, also known as binary cross-entropy, with wins weighted by the ratio of losses to wins to offset class imbalance. The optimization technique is gradient boosting; each new tree is fit to the first and second derivatives of the loss and added to the ensemble with a learning rate shrinkage factor. Regularization is performed through row and column subsampling and shallow trees. Because of the weighting, outputs are scores rather than calibrated probabilities.

LightGBM Status: LightGBM, another gradient-boosting framework, is implemented in the project using the same data, split, features, loss weighting, purging and metrics as XGBoost, plus its own 36-setting grid over number of leaves, learning rate, number of trees and minimum leaf size. The similarities in implementation are intended to provide a meaningful comparison to XGBoost results. However, while the code for its use exists, LightGBM has not yet been trained on any data, meaning that it cannot provide any results at this point. LightGBM results are expected to be included in a final project report.

Tuning Hyperparameters: For tuning of hyperparameters, we searched a 24-setting grid with tree depth of 2, 3 or 4, learning rate of 0.03 or 0.1, 100 or 300 trees, and a minimum child weight of 5 or 20. Each setting was scored by mean PR-AUC over expanding-window walk-forward folds in the training data. PR-AUC was used because of its fit for imbalanced labels.

Baseline and Results: The classification baselines are comparing against majority vote and chance. For trading, we compare against taking every setup the detector finds, the success criterion of our proposal, and against buy-and-hold of SPY, QQQ and an equal-weight basket of all stocks. The baseline of majority vote reaches 77.8% accuracy on the test set (207 losses out of 266 predictions). At a threshold of 0.5, XGBoost reaches 56.4% accuracy, with the difference indicating that accuracy on its own is uninformative. As a consequence, we compared precision and AUC with their chance levels. Precision was 20.6% (slightly below the 22% base rate), recall was 33.9%, and the true skill statistic was about -0.03. Test ROC-AUC was around 0.476 against 0.5 for chance. Taking every setup earned roughly +0.10 R per trade over the test period against roughly +0.05 R for the 97 setups the model picked. Buy-and-hold SPY and QQQ returned 0.03 and -0.0322 over the same dates, respectively. A walk-forward evaluation, which retrains before each of seven windows and produces about 1,000 out-of-sample predictions, gives per-window ROC-AUC between about 0.39 and 0.65, centred on 0.5. With the current features, the model thus cannot be seen to give any reliable signal, which is consistent with the noise anticipated in the project proposal.

Ablation: To see which feature groups are most relevant, we repeated the walk-forward evaluation with detector features only, detector plus technical, detector plus macro, and all features. None stayed above chance across all windows. Detector plus macro proved best in three of the middle windows (ROC-AUC 0.56 to 0.64) but fell below 0.5 in others. Pooled over all 1,015 predictions, ROC-AUC is 0.513 (95% interval 0.470 to 0.555) for detector only, 0.502 (0.458 to 0.546) with technical features, 0.517 (0.474 to 0.561) with macro features and 0.519 (0.474 to 0.562) with all features, so every interval includes 0.5.We also tested relative-strength features (each stock against SPY and against the other tickers) and dropped them because they did not help. Feature importances are flat, with macro features generally being best. Since macro values are identical for all setups at the same moment, this reflects how the trees split the data and is not evidence that these features predict outcomes.

6 Results and Evaluation

Splitting: Setups are sorted by signal time. The last 20% (268 setups) is the final test set and the first 80% (1,038 setups) is used for training. A random split would let the model learn from setups that happen at the same time as the test setups, which inflates results. We also use a 3-day embargo, so training trades have to close at least 3 days before the test window starts. This means trade outcomes do not overlap with the test period.

Cross-validation: Hyperparameters are only tuned on the training set, they use expanding-window walk-forward folds.

- The training data is split into 5 chunks in time order.
- Fold k trains on chunks 1 to k and validates on chunk $k + 1$, with the same embargo.
- Settings are picked by the average PR-AUC across the folds.

To have a stronger check, a full walk-forward run retrains the model across 8 time windows over the whole history and scores each window on data it has not seen.

Classification metrics: Only about 21% of test setups are wins, which makes it inaccurate. These get reported:

Test set (268 setups)	XGBoost	LightGBM
ROC-AUC	0.49	0.46
PR-AUC	0.21	0.20
Precision	21.1%	18.8%
Recall	26.8%	26.8%
TSS	0.004	−0.04

Table 2: Classification result for the final test set. The base rate is 20.9% wins.

- precision, which is the amount of predicted wins that actually win
- recall, which is the share of actual wins that the model predicts as wins
- PR-AUC, compared against the 21% base rate
- ROC-AUC, 0.5 means random ranking
- TSS (true skill statistic), which is recall minus the false positive rate, where 0 means no skill

Trading metrics: Trades measured in R, which is profit in units of the risk taken. A full win is +2R and a full stop out is −1R. We report win rate, average R per trade, Sharpe ratio and max drawdown. To compare with buy-and-hold percentage returns, each trade is assumed to risk 1% of the account.

Baselines:

- buy and hold SPY and QQQ
- equal weight buy and hold of all 76 tickers
- the unfiltered detector takes every setup

The unfiltered detector is the main baseline. If the model does not beat it, the model does not add value.

Results so far: Table 2 shows the test set results. Neither model decides winners against losers better than chance, and both have precision almost equal to the base rate.

While tuning, the best settings scored about 0.30 PR-AUC, compared to fold base rates of 0.20 to 0.27. This looked like a small edge, but it disappeared on the test set. Picking the best of 24 (XGBoost) or 36 (LightGBM) settings on only about 1,000 rows gives a score that is good but only because of luck. Both models also picked the largest minimum leaf size, which is what tuning tends to do when there not much of a real pattern to find.

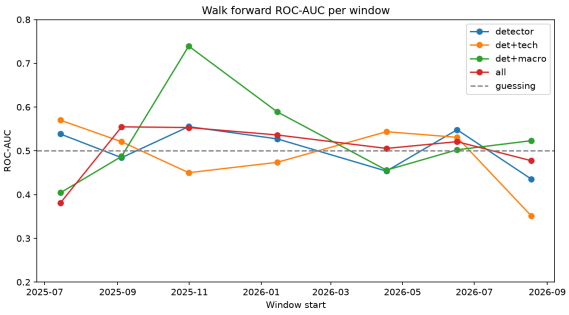


Figure 1: Walk-forward ROC-AUC for each time window and feature set, the values stay close to 0.5.

	Return	Sharpe	Max DD	Trades
SPY hold	+3.5%	0.91	3.5%	–
QQQ hold	−4.5%	−0.53	10.3%	–
Equal-wt hold	+9.9%	2.11	3.3%	–
Unfiltered detector	+4.1%	0.47	18.8%	268
XGBoost ($p \geq 0.5$)	+2.2%	0.43	7.8%	71

Table 3: Backtest over the final test window. Each trade risks 1% of the account. The equal weight hold is rebalanced every day.

The walk-forward results are similar (Figure 1). Across both models and all the feature sets, ROC-AUC is between 0.49 and 0.53, and every 95% confidence interval has 0.5. ROC-AUC for individual windows ranges from 0.35 to 0.74 where there s no steady trend, which indicates luck and not a skill that will hold over time.

Table 3 shows the backtest over the approximately 90-day test window.

- The XGBoost filter took 71 of the 268 setups.
- It returned +2.2% (Sharpe 0.43), compared to +4.1% (Sharpe 0.47) for the unfiltered detector.
- It reduced the max drawdown from 18.8% to 7.8%, mostly because it took fewer trades.
- The equal weight buy and hold returned +9.9% (Sharpe 2.11), which is higher than both.

The model's picks earned a little more per trade than all the setups on the test set, 0.041R compared to 0.027R (Figure 2). With ROC-AUC around 0.5, it is likely that the difference is due to noise. Total R was also lower for the model (2.9R vs 7.2R) because it took less trades.

Over the full history, the unfiltered detector averaged +0.157R per trade over 1,315 trades. This suggests the pullback pattern is mildly positive on



Figure 2: Cumulative R for model filtered trades compared to all of the detector setups.

its own, but our current features do not predict which setups will work.

7 Feedback and Plans

TA Feedback and What We Learned: We received feedback on the current state of our project from a TA. We were told to look at more complex classifiers, different features, and different weights in order to make our model more accurate. It was additionally suggested that accuracy was not essential so long as we could explain the current lack of accuracy and articulate future steps to remedy it. At the moment, we have identified some explanations as to why the model is not very accurate based on our results so far, which have generally demonstrated a performance no better than chance: these include a noisy target (which was anticipated), a model that overfits, and weak features.

Remaining Timeline: The timeline of the project remains broadly unchanged: we plan to finish the training and implementation of LightGBM, build the trade simulation engine (which we hope will replace the fixed-risk approximation in our baselines), obtain backtest results, and finish the project by performing testing and ensuring that the various components of the project are integrated well. However, if we wish to attain higher accuracy, certain finer details must change.

Changes and What We Will Do Differently: Based on the feedback and the fact that we have thus far struggled to attain a model that performs well, we will attempt to make a number of changes in the specifics how we proceed. We will try to look over features to remove any redundant ones and add more setup-specific ones in the hope that this improves model performance. We also intend to test class weights beyond the current ratio and choose the decision threshold on validation folds. Another idea for future change is judging models by walk-forward results with confidence intervals rather than the current setup test split in order to potentially reduce noise and the chance of overfitting. We do not immediately intend to increase model complexity, but may do so if these fixes do not produce improvement. Implementation thus far has gone smoothly, with only the results appearing to disappoint, so we hope that these adjustments to the model can produce better results.

Limitations

One limitation is that the Yahoo data covers only two years, which means we have a single market condition, an uptrend. The test period was also short (approximately 90 days with 268 setups), so the margin of error is big, as we have ROC-AUC 95% confidence intervals of approximately ± 0.08. To account for this, we backtested the whole history of the stocks and got about 1,000 out-of-sample predictions. We did not account for commissions, slippage, or position limits, and set the risk per setup to 1R, which is 1% of the account in the backtest. We chose 76 tickers, considering the availability of Yahoo Finance and GDELT data, which may bias the samples towards stocks that are more often covered by these sources. We also considered several combinations of features, target values, and models, which increases the risk of a multiple comparison problem, where a model may appear statistically significant even if it is not.

Team Contributions

Below is a summary of each team member's work since the proposal:

Kelsey set up the GitHub repository, implemented the OHLCV data pull and resampling to 4-hour bars, wrote the setup detector and triple barrier labeler, and wrote the tests for them. Kelsey also implemented the shared leakage detection, which was run on all of the macro features. She also trained and tuned XGBoost on a purged time split, including the full-history evaluation step, the backtest metrics such as win rate, Sharpe ratio, and maximum drawdown, and the result visualization including the dashboard for per-setup model scores.

Rishi created the GDELT and VIX macro data pipeline using BigQuery aggregation, caching, and

macro features, and merged them with setups. He also wrote the macro leakage check script, and completed the LightGBM training.

Rhea set up access to the GDELT dataset through BigQuery, implemented an indicator library (including EMA, SMA, ATR, RSI, and MACD), implemented technical features such as MACD, volatility, volume and their leakage checks. She was also responsible for dataset compilation and cleaning, detector output comparison for ticker groups and definition of baseline strategies such as the buy-and-hold and the unfiltered detector.

AI Disclosure

Throughout the project up to this point our team has used a couple of AI models such as Claude, Google's Gemini and ChatGPT to learn more about the specific models and concepts used in our project. We used them to explain more about XGBoost and LightGBM functionality, complex parts of the feature engineering and our result interpretation. We also used them to help extract numbers from our plots of the results in the baselines section, which have been verified against our own work. For code implementation, Claude was used to debug the code and help with some code reviews, as well as to suggest implementation approaches and fixes for the nested walk-forward evaluation with purged and embargoed time splits, and the triple-barrier labeler. Claude was also used for formatting LaTeX tables and the report document to render it correctly. The team has made sure to thoroughly review, understand, and validate any AI-generated information or code, to ensure it fits with the project and our understanding of it. Gemini and ChatGPT were used mostly for verifying Claude's answers and comparing their results with each other. Overall, the AI models used include Claude Sonnet 5 for roughly 30 prompts ($\sim$9.6 g CO_2), Google's Gemini 3 for roughly 5 prompts ($\sim$0.65 g CO_2), and ChatGPT GPT-5.6 Luna for roughly 7 prompts ($\sim$1.06 g CO_2) (Devera Team, 2026).

References

Suryoday Basak, Saibal Kar, Snehanshu Saha, Luckyson Khaidem, and Sudeepa Roy Dey. 2019. Predicting the direction of stock market prices using tree-based classifiers. *The North American Journal of Economics and Finance*, 47:552–567.

Devera Team. 2026. AI water & energy usage: ChatGPT vs Claude vs Gemini (2026). Devera. Accessed Oct. 6, 2026.

Christopher Krauss, Xuan Anh Do, and Nicolas Huck. 2017. Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2):689–702.

Marcos López de Prado. 2018. *Advances in Financial Machine Learning*. Wiley, Hoboken, NJ.

Paul C. Tetlock. 2007. Giving content to investor sentiment: The role of media in the stock market. *The Journal of Finance*, 62(3):1139–1168.

Hanyuan Zhang, Yankai Wang, Zexuan Li, and Xiaoyin Wang. 2025. Machine learning models for bank customer churn prediction: A comparative study of lightgbm, catboost, and xgboost. In *Proceedings of the 2025 International Conference on Big Data, Artificial Intelligence and Digital Economy*, BDAIE '25, page 6–16, New York, NY, USA. Association for Computing Machinery.